

Tema 7 — Gestión de errores

Contenido Teórico

Objetivos

- **Entender qué es una excepción** y por qué aparece durante la ejecución de un programa.
- **Diferenciar errores** de sintaxis, errores de ejecución y errores de lógica.
- **Usar try, except, else y finally** para controlar fallos previsibles.
- **Capturar excepciones específicas** como ValueError, KeyError, IndexError o ZeroDivisionError.
- Lanzar errores con raise y crear excepciones personalizadas sencillas.
- **Aplicar buenas prácticas** para diagnosticar sin ocultar errores importantes.

Mapa del Tema 7

- Primero veremos qué ocurre cuando Python encuentra un error en tiempo de ejecución.
- Después estudiaremos try/except, múltiples excepciones, else y finally.
- A continuación trabajaremos raise, excepciones personalizadas y diagnóstico básico.
- El objetivo no es envolver todo el código en try, sino controlar puntos donde el fallo es razonablemente previsible.
- Flujo general: entrada -> validación -> operación -> excepción -> diagnóstico o recuperación.

Qué es una excepción

Una excepción es una situación anómala que interrumpe el flujo normal del programa.

- Python crea un objeto de excepción con información sobre el problema ocurrido.
- Si no se captura, el programa termina y muestra una traza de error.

Gestionar errores permite responder de forma controlada: informar, usar un valor alternativo, registrar el fallo o detener con un mensaje claro.

```
puerto = "ssh"

# ValueError: no puede convertir texto no numérico
numero = int(puerto)
print(numero)
```

Tipos de errores comunes: sintaxis, ejecución y lógica

- **Error de sintaxis:** el código no es válido y Python no puede interpretarlo.
- **Error de ejecución:** el código es válido, pero falla al ejecutarse con ciertos datos.
- **Error de lógica:** el programa se ejecuta, pero el resultado no es el esperado. Suelen ser más difíciles de identificar.

try/except controla errores de ejecución, pero no arregla código mal escrito ni rectifica decisiones incorrectas de diseño.

```
# Sintaxis: falta cerrar paréntesis
print("Hola"

# Ejecución: clave inexistente
servicio = {"nombre": "ssh"}
```

```
print(servicio["puerto"])

# Lógica: puerto incorrecto
puerto_https = 80
```

Excepciones frecuentes en scripts

- **SyntaxError**: error de sintaxis en el script.
- **TypeError**: uso de un tipo de dato en una operación no compatible. Por ejemplo, sumar un número y una cadena.
- **ValueError**: el valor no tiene sentido para una operación. Por ejemplo, `int("abc")`.
- **KeyError**: clave inexistente en un diccionario.
- **IndexError**: índice fuera de rango en lista o tupla.
- **ZeroDivisionError**: división entre cero.
- **FileNotFoundError**: fichero o ruta inexistente.
- **PermissionError**: permisos insuficientes para acceder a un recurso.

```
datos = {"ssh": 22}
servicios = ["ssh", "nginx"]
numero = "10"

# SyntaxError (if sin :)
# if True

# TypeError (suma cadena y número)
# suma = numero + 10

# ValueError ("abc" no se puede convertir)
# suma = int("abc") + 20

# KeyError
# print(datos["api"])

# IndexError
# print(servicios[5])

# ZeroDivisionError
# print(10 / 0)
```

Flujo de una excepción y try/except básico

- Python ejecuta las instrucciones en orden. Cuando una instrucción falla, se crea una excepción y se abandona el bloque actual.
- Con **try/except** podemos acotar un bloque que potencialmente puede fallar.
 - El bloque **try** contiene la operación que puede fallar. El bloque **except** contiene la respuesta ante el error que podría ocurrir.
 - En caso de fallo, Python busca un bloque **except** con un tipo de error compatible. Si no lo encuentra, la excepción se propaga y el programa termina.
- El código que puede fallar debe ser lo más pequeño posible. Así evitamos capturar errores que no tienen relación con la operación que queremos controlar.

```
print("Inicio")

try:
    numero = int("abc")
    print("No se ejecuta")
except ValueError:
    print("Dato no convertible")

print("Fin")
```

Captura de excepciones específicas y orden de los bloques

Python prueba los bloques except de arriba abajo.

- Conviene capturar primero los errores concretos que podemos esperar. Las excepciones específicas permiten responder de forma mucho más precisa y deben ir antes que las excepciones genéricas.
- **except Exception** captura casi cualquier error normal de ejecución, pero no captura **KeyboardInterrupt** ni **SystemExit**.
- Si **Exception** se coloca primero, los bloques específicos posteriores no se ejecutarán nunca.
- Capturar **Exception** sin criterio puede ocultar problemas reales del programa. Un **except** vacío captura cualquier excepción y es todavía más peligroso.

```
dividendo = "10"
# divisor = "0"      # ZeroDivisionError
divisor = "2"
# divisor = "dos"    # ValueError

try:
    dividendo = int(dividendo)
    divisor = int(divisor)
    resultado = dividendo / divisor
    print(int(resultado))
    # print(int(noexiste)) # NameError gestionado por Exception
except ValueError:
    print("Conversión inválida")
except TypeError:
    print("El divisor no es numérico")
except ZeroDivisionError:
    print("No se puede dividir por cero")
except Exception:
    print("Error general")
```

No capturar todo sin criterio: uso de 'as'

- Un **except** vacío o demasiado genérico puede ocultar errores no previstos.
- Si se captura **Exception**, debe existir una razón clara y normalmente se debe registrar el detalle.
- Evitar mensajes demasiado genéricos ayuda al diagnóstico. Una excepción silenciada puede convertir un fallo evidente en un comportamiento incorrecto difícil de localizar.
- **except** puede guardar el objeto de excepción con **as**. Esto permite consultar el mensaje en crudo generado por Python.
- En soporte puede combinarse un mensaje claro al usuario junto con el mensaje técnico si aporta valor.

```
valor = "https"

try:
    puerto = int(valor)
    print(puerto)
except ValueError as error:
    print("Puerto no válido")
    print("Detalle del error:", error)

# Evitar
# try:
#     puerto = int("ssh")
# except:
#     print("Fallo general")
```

Capturar varias excepciones juntas

Cuando la respuesta es la misma para varios errores, pueden agruparse en una tupla.

- No conviene agrupar errores si necesitan diagnósticos o acciones diferentes.
- La tupla de excepciones mantiene el código compacto sin perder claridad.
- También se puede usar **as error** para consultar el detalle concreto del error producido.

```
datos = ["22", "abc"]
indice = 2

try:
    puerto = int(datos[indice])
    print(puerto)
except (ValueError, IndexError) as error:
    print("No se pudo obtener el puerto")
    print(type(error).__name__, error)
```

Bloques else y finally

- **else** se ejecuta solo si el bloque try termina sin excepciones. Permite separar la operación peligrosa del código que solo debe ejecutarse cuando todo fue bien.
- **finally** se ejecuta siempre, haya error o no. Se usa para liberar recursos o ejecutar limpieza final una vez terminado el flujo.

En gestión de ficheros, conexiones o bloqueos, finally ayuda a cerrar lo que se abrió al principio del código.

No es obligatorio usar todos los bloques.

- **try** contiene la operación susceptible de fallar
- **except** responde al error
- **else** contiene el camino correcto
- **finally** contiene la limpieza final.

```
valor = "443"
puerto_abierto = True

try:
    puerto = int(valor)
except ValueError:
    print("Puerto no válido")
```

```
print("Puerto abierto:", puerto_abierto)
else:
    print(f"Puerto aceptado: {puerto}")
    print("Puerto abierto:", puerto_abierto)
finally:
    puerto_abierto = False
    print("Puerto abierto:", puerto_abierto)
```

La decisión: validar antes o capturar después

En programación existen dos alternativas habituales para gestionar fallos previsibles.

- **LBYL**, Look Before You Leap, consiste en comprobar una condición antes de operar. Puede duplicar consultas: primero se comprueba si una clave existe y después se accede a ella.
- **EAFP**, Easier to Ask for Forgiveness than Permission, consiste en intentar la operación y capturar el fallo. En Python es un enfoque habitual cuando mejora la claridad.

También existen métodos de acceso seguro, como **get()** en diccionarios, que permiten devolver un valor por defecto cuando la clave no existe.

La decisión final depende de la legibilidad del código, el coste de la comprobación y el tipo de recurso.

```
puertos = {"ssh": 22, "nginx": 443}
servicio = "api"

# Enfoque LBYL: comprobar antes
if servicio in puertos:
    print(puertos[servicio])
else:
    print("Servicio no registrado")

# Enfoque EAFP: capturar después
try:
    print(puertos[servicio])
except KeyError:
    print("Servicio no registrado")

# Acceso seguro con valor por defecto
print(puertos.get(servicio, "Servicio no registrado"))
```

Lanzar excepciones con raise

raise permite provocar una excepción de forma explícita.

- Se usa cuando detectamos un estado inválido que la función no debe aceptar.
- Es preferible lanzar un error claro a devolver valores ambiguos.
- El código que llama a la función decide si captura el error o deja que el programa termine.

```
def validar_puerto(puerto):
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return puerto

print(validar_puerto(443))
# print(validar_puerto(70000))
```

Validación dentro de funciones

Una función puede validar sus argumentos antes de continuar.

- Si recibe datos incorrectos, debe fallar con un mensaje claro.
- Esto evita errores posteriores más difíciles de diagnosticar.
- La validación formal complementa las anotaciones de tipo en la definición de la función. Las anotaciones ayudan, pero no impiden valores inválidos por sí solas.

```
def crear_endpoint(servicio: str, puerto: int) -> str:
    if not servicio:
        raise ValueError("Servicio vacío")
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return f"{servicio}:{puerto}"

print(crear_endpoint("ssh", 22))
# print(crear_endpoint("", 22))
# print(crear_endpoint("ssh", 70000))
```

Validación al procesar colecciones

En automatización se procesan listas de datos donde algunos elementos pueden ser incorrectos.

- No siempre interesa detener todo el proceso por un único registro defectuoso.
- **try/except** dentro de un bucle permite separar valores válidos e inválidos.
- Conviene guardar también qué dato falló para poder revisarlo después.

```
def validar_puerto(puerto):
    if not 1 <= puerto <= 65535:
        raise ValueError("Puerto fuera de rango")
    return puerto

datos = ["22", "ssh", "443", "70000"]
validos = []
rechazados = []

for valor in datos:
    try:
        validos.append(validar_puerto(int(valor)))
    except ValueError as error:
        rechazados.append((valor, str(error)))

print(validos)
print(rechazados)
```

Relanzar una excepción

A veces se captura un error solo para añadir contexto o registrar información.

- Después puede relanzarse con `raise` para no ocultarlo.
- Relanzar conserva el flujo de error y evita continuar en un estado inseguro.
- Debe usarse cuando el nivel actual no puede resolver realmente el problema.

```
def convertir_puerto(valor):
    try:
        return int(valor)
    except ValueError:
        print(f"Fallo al convertir: {valor}")
        raise

convertir_puerto("22")
# convertir_puerto("ssh")
```

Excepciones personalizadas

Una excepción personalizada representa un error propio del dominio del programa.

- Como las excepciones son clases, se declara mediante herencia.
- Debe heredar de `Exception` o de una excepción más específica. Se lanza con `raise` y se captura con `except`.
- Conviene usar nombres terminados en `Error` y no crear una excepción personalizada para cada fallo menor.

El código que llama puede capturar la excepción personalizada de forma específica. Esto permite diferenciar **errores de negocio** de **errores técnicos generales**.

```
class ServicioNoDisponibleError(Exception):
    pass

def comprobar_servicio(nombre, activo):
    if not activo:
        raise ServicioNoDisponibleError(
            f"{nombre} no está activo"
        )
    return "OK"

try:
    estado = comprobar_servicio("api", False)
except ServicioNoDisponibleError as error:
    print("Incidencia de servicio")
    print(error)
else:
    print(estado)
```

Diagnóstico con traceback

Una traza o **traceback** muestra el camino de llamadas hasta el punto donde apareció el error.

- Cuando se produce un error, la traza se muestra con el mensaje **Traceback (most recent call last)**.
- Es útil para depuración y soporte, pero puede ser demasiado técnica para un usuario final.
- El módulo **traceback** permite imprimir información detallada cuando estamos diagnosticando. En producción suele preferirse registrar el detalle y mostrar un mensaje controlado.

```
import traceback

try:
    int("ssh")
except ValueError as error:
```

```
print("Error controlado:", error)
traceback.print_exc()
```

Documentar errores para comunicar contratos

Una función, clase u otro componente establece un acuerdo de uso entre quien lo escribe y quien lo utiliza (Contrato).

Una función tiene un contrato: qué recibe, qué devuelve y qué errores puede lanzar.

El docstring de la función debe indicar casos relevantes de error cuando no sean evidentes.

Esto ayuda a otros programadores a usar la función correctamente. En código de entornos TI, un contrato claro reduce fallos encadenados.

```
def obtener_puerto(config, servicio):
    """Devuelve el puerto de un servicio.

    Raises:
        KeyError: si el servicio no existe.
    """
    return config[servicio]

puertos = {"ssh": 22}
print(obtener_puerto.__doc__)
print(obtener_puerto(puertos, "ssh"))
# print(obtener_puerto(puertos, "api")) # KeyError
```

Buenas prácticas y errores frecuentes

Buenas prácticas:

- Capturar el error más específico posible
- Mantener el bloque try pequeño
- Usar else para el camino correcto cuando mejore la lectura
- Usar finally para limpieza obligatoria
- Incluir mensajes claros y datos relevantes para diagnóstico
- No ocultar errores que el programa no sabe resolver.

Errores frecuentes:

- Capturar **Exception** en todas partes sin necesidad
- Usar **except: pass**
- Meter demasiado código dentro del try
- Capturar errores específicos después de Exception
- Mostrar trazas técnicas al usuario final sin filtrar
- Usar excepciones para controlar casos normales que podrían validarse de forma clara.

```
# Buena práctica
try:
    puerto = int(valor)
except ValueError as error:
    print(f"Puerto inválido: {valor}")
else:
    print(f"Puerto válido: {puerto}")
finally:
```



```
    print("Comprobación completada")

# Error: demasiado amplio
try:
    ejecutar_tarea()
except Exception:
    print("Algo falló")

# Más útil
try:
    puerto = int(valor)
except ValueError:
    print("Puerto no numérico")
```

Resumen del tema

- Las excepciones permiten controlar fallos previsibles en tiempo de ejecución.
- try/except no sustituye al diseño correcto ni a la validación de datos.
- else y finally ayudan a separar camino correcto y limpieza final.
- raise permite informar de estados inválidos de forma explícita.
- Las excepciones personalizadas aportan claridad cuando representan errores propios del dominio.
- La meta es que el programa falle de forma comprensible y diagnosticable.

```
try:
    ejecutar()
except ErrorEsperado as error:
    diagnosticar(error)
else:
    continuar()
finally:
    limpiar()
```

Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema07.
- Primero se trabajará con bloques try/except/else/finally.
- Después se aplicará raise y gestión de errores personalizada, incluyendo validación de errores y diagnóstico con traceback.
- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python/Tema07
source .venv/bin/activate
python --version

# Actualizar del repositorio Git
git pull origin main

# Para ejecutar VS Code
code . # usar carpeta Tema07

# Para ejecutar en terminal
python
```